
Objet de TP

l'objet du TP est d'explorer et d'évaluer la méthode de rejet pour simuler des distributions de probabilité, en mettant en lumière à la fois ses avantages et ses limites.

Question 1

```
gcl<-function(a,b,m){  
  x<-(a*x+b)%m  
  
  return(x)  
}  
  
m=1171*8*7*9  
a=1171*4*7*3+1  
b=11*13*17*19  
  
x<-789547  
Seq1<-replicate(100,gcl(a,b,m))  
UniSeq1<-Seq1/(m-1)  
x<-25794  
Seq2<-replicate(100,gcl(a,b,m))  
UniSeq2<-Seq2/(m-1)
```

Dans la première question, nous avons généré une séquence de 100 nombres pseudo-aléatoires en utilisant comme point de départ la graine $x_0 = 789547$ et en utilisant un générateur de nombres congruents linéaires (GCL) avec les paramètres suivants : $a = 1171 \times 4 \times 7 \times 3 + 1$, $b = 11 \times 13 \times 17 \times 19$, et $m = 1171 \times 8 \times 7 \times 9$.

Question 2

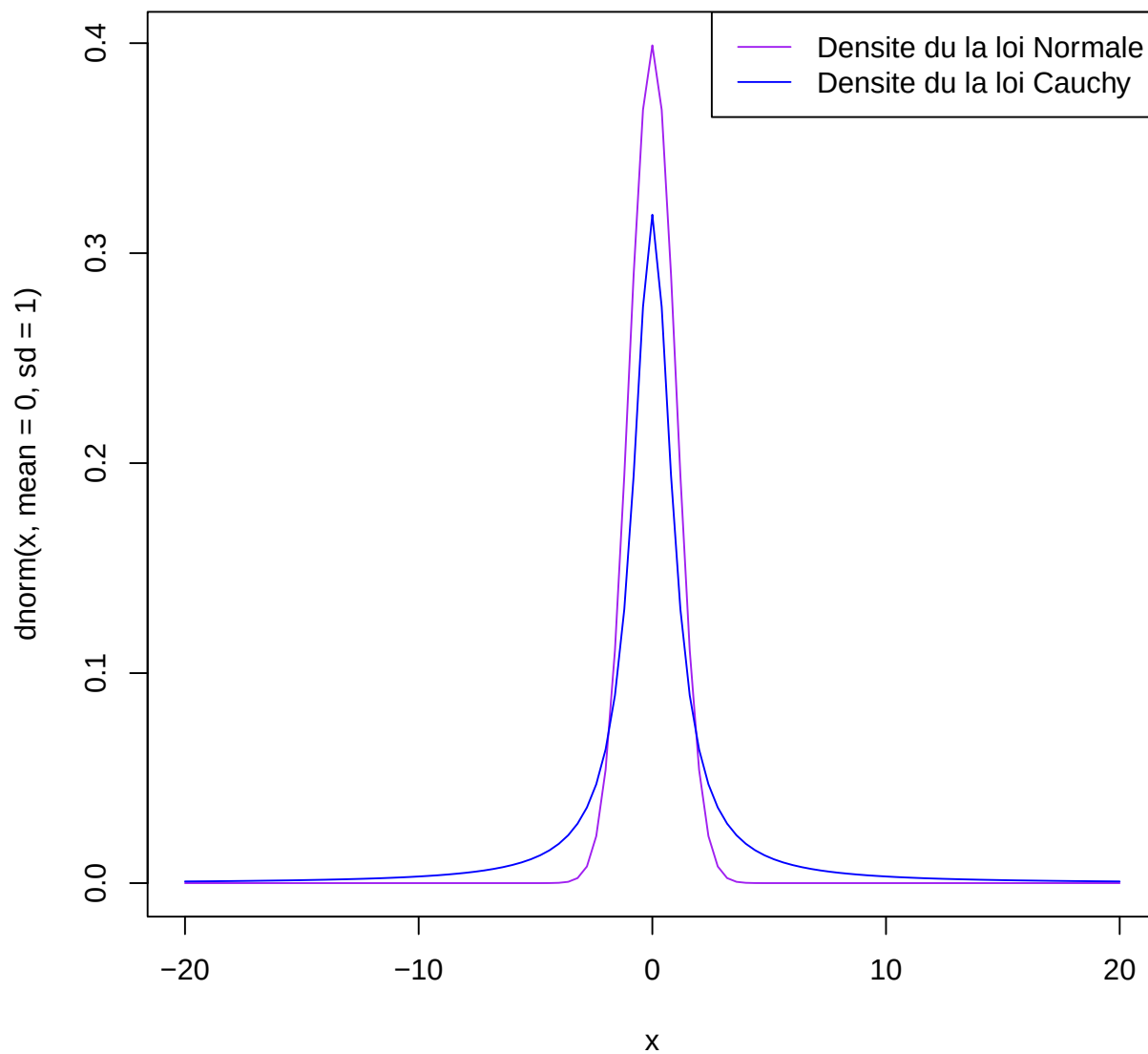
```
x<-25794  
Seq2<-replicate(100,gcl(a,b,m))
```

Dans la deuxième question, une autre séquence de 100 nombres pseudo-aléatoires a été générée en utilisant comme point de départ la graine $x_0 = 25794$ avec le même générateur de nombres congruents linéaires (GCL) que précédemment.

Question 3

```
curve(dnorm(x,mean = 0,sd=1), from = -20, to = 20, col = "purple", lwd = 1)

curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
legend("topright", legend = c("Densite du la loi Normale", "Densite du la loi Cauchy"),
      col = c("purple", "blue"), lty = 1, lwd = 1)
```



```
(MaxNormal<-dnorm(0, mean = 0,sd=1))
## [1] 0.3989423

(MaxCauchy<-dcauchy(0,0,1))
## [1] 0.3183099
```

D'après le graphe, nous observons que le maximum des densités de probabilité des deux lois, la loi normale $\mathcal{N}(0, 3)$ et la loi de Cauchy avec des paramètres $a = 0$ et $x_0 = 0$, se trouve à l'indice $x_0 = 0$.

De plus, l'apogée des deux fonctions est atteinte lorsque $x = 0$, ce qui correspond au point où les courbes culminent.

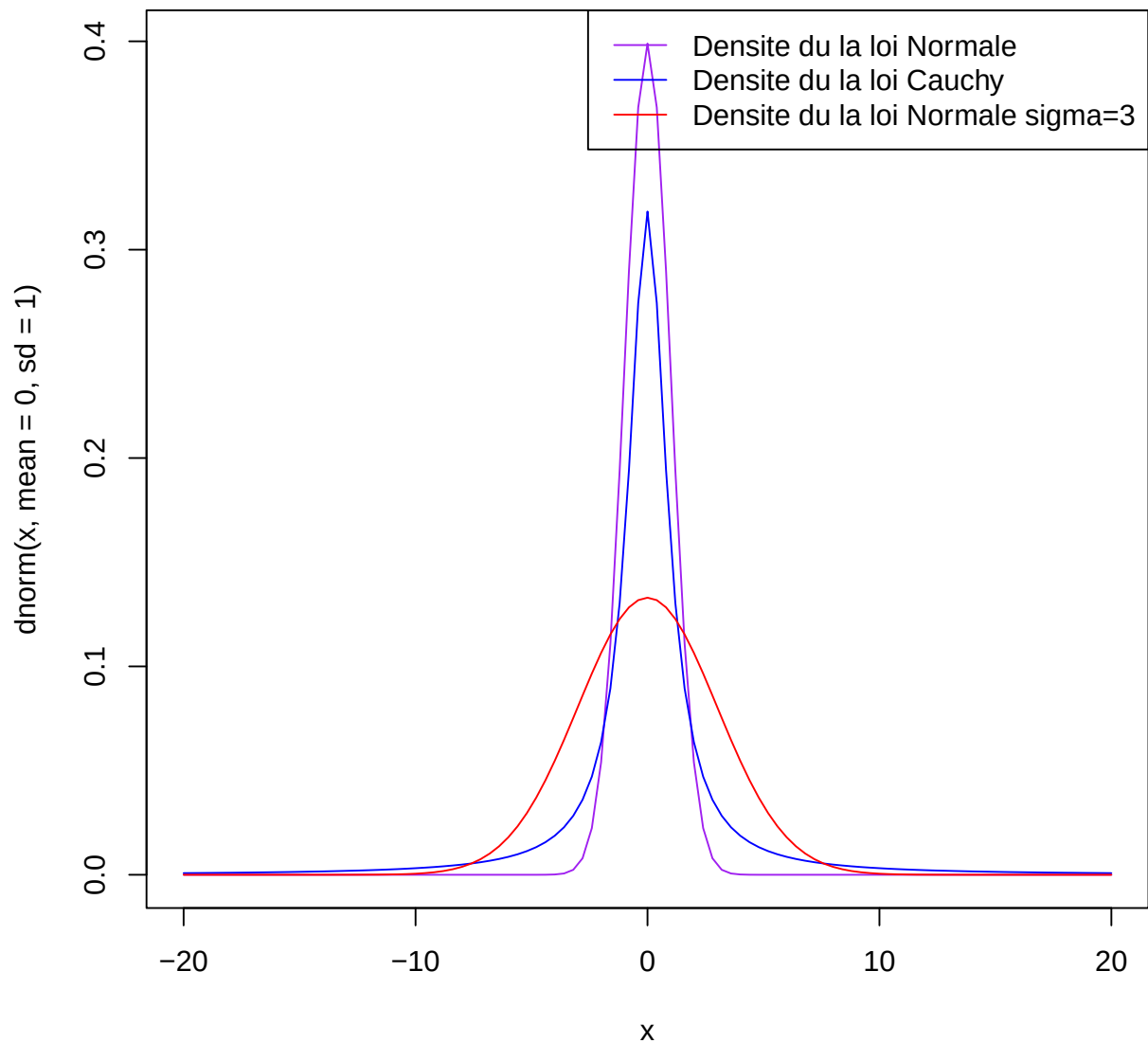
Cela signifie que la densité de probabilité est maximale pour ces deux distributions lorsque la variable aléatoire prend la valeur zéro.

- `dnorm` et `dcauchy` : Ce sont des fonctions de densité de probabilité. Elles calculent la densité de probabilité pour une variable aléatoire suivant une distribution normale ou une distribution de Cauchy, respectivement.
- `curve` : Cette fonction permet de tracer des courbes dans un graphique. Elle prend comme argument une expression R qui décrit la courbe à tracer. Dans ce cas, nous utilisons `dnorm` et `dcauchy` pour tracer les densités de probabilité des lois normale et de Cauchy respectivement. Les arguments `from` et `to` spécifient les limites sur l'axe des x où tracer la courbe. L'argument `add = TRUE` permet d'ajouter la courbe à un graphique existant.
- `legend` : Cette fonction permet d'ajouter une légende au graphique. Elle prend des arguments pour spécifier la position de la légende, les noms des éléments à légender, ainsi que les couleurs, styles de ligne, etc.

Question 4

```
sd=3
curve(dnorm(x,mean = 0,sd=1), from = -20, to = 20, col = "purple", lwd = 1)

curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
curve(dnorm(x,mean = 0,sd=sd), from = -20,add = TRUE, to = 20, col = "red", lwd = 1)
legend("topright", legend = c("Densite du la loi Normale", "Densite du la loi Cauchy",
"Densite du la loi Normale sigma=3"), col = c("purple", "blue","red"), lty = 1, lwd = 1)
```



```
(MaxNormal3<-dnorm(0,0,3))
```

```
## [1] 0.1329808
```

```
(MaxNormal3>=MaxCauchy)
```

```
## [1] FALSE
```

D'après le graphique, on observe que la densité de probabilité de la loi normale $\mathcal{N}(0,3)$ est significativement plus petite que le maximum de la densité de probabilité de la loi de Cauchy avec des paramètres $a = 0$ et $x_0 = 0$.

```
M=1
```

```
while (M*MaxNormal3<=MaxCauchy) {
```

```
  M<-M+0.1
```

```
}
```

```
cat("la constante est egal:",M)
```

```
## la constante est egal: 2.4
```

Nous avons initialisé M avec une valeur de 1 dans le but d'augmenter le sommet de la densité de probabilité de la loi normale $\mathcal{N}(0, 3)$.

Nous avons choisi cette valeur initiale car nous voulions éviter de diminuer la densité de probabilité lorsque nous multiplions par M .

En incrémentant M de 10^{-1} , la boucle se répète jusqu'à ce que le maximum de la densité de probabilité de la loi normale $\mathcal{N}(0, 9)$ atteigne celui de la loi de Cauchy avec les paramètres $a = 0$ et $x_0 = 0$.

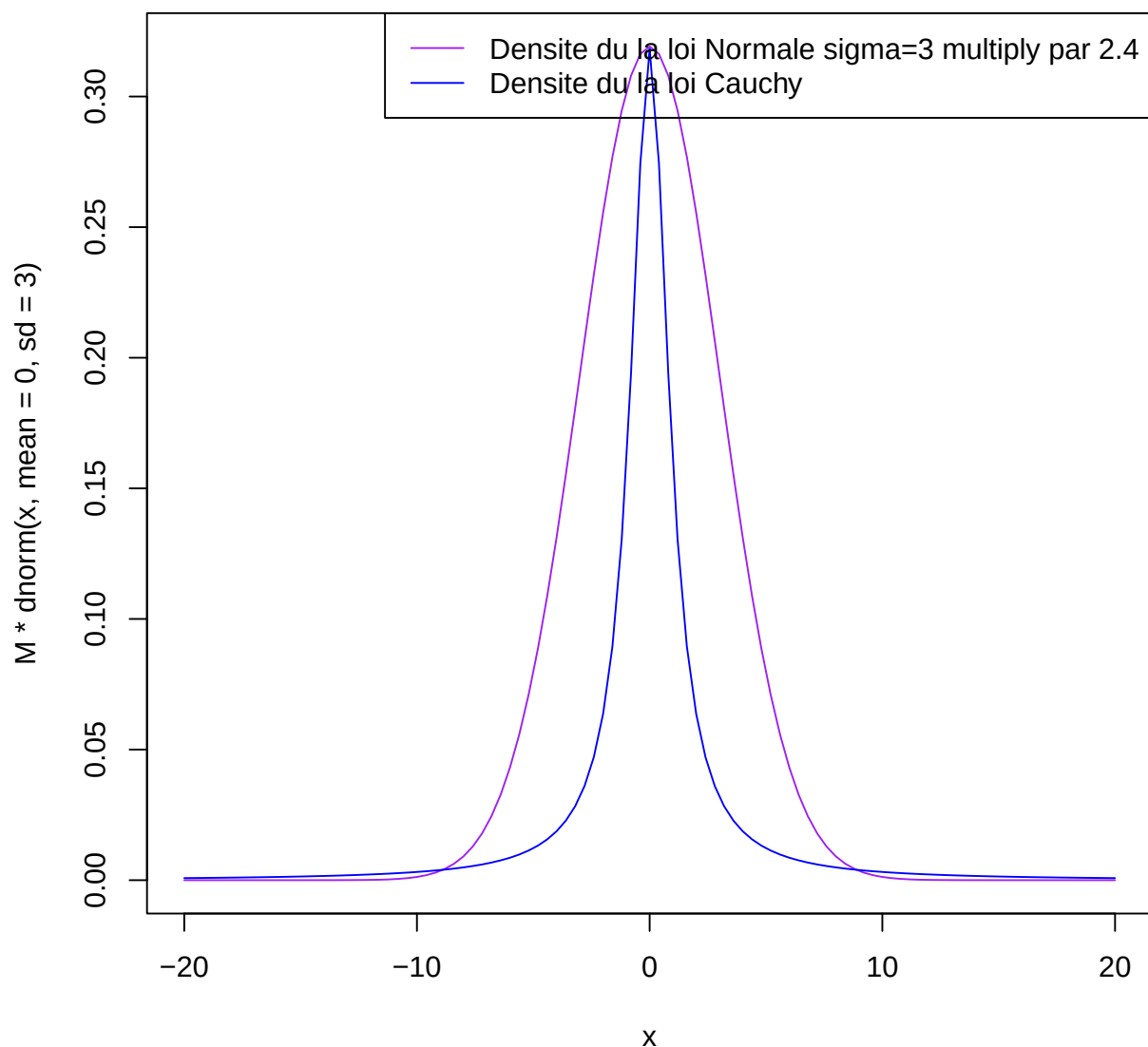
```
(M*MaxNormal3>=MaxCauchy)
```

```
## [1] TRUE
```

Nous confirmons cette inégalité $M \times \text{MaxNormal3} \geq \text{MaxCauchy}$, qui indique que le maximum de la loi $\mathcal{N}(0, 3)$ est supérieur ou égal au maximum de la loi de Cauchy avec les paramètres $a = 0$ et $x_0 = 0$, comme le montre le graphique ci-dessous.

```
curve(M*dnorm(x,mean = 0,sd=3), from = -20, to = 20, col = "purple", lwd = 1)

curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
legend("topright", legend = c("Densite du la loi Normale sigma=3 multiply par 2.4",
"Densite du la loi Cauchy"),
      col = c("purple", "blue"), lty = 1, lwd = 1)
```



```
(dcauchy(-10,0,1))
```

```
## [1] 0.003151583
```

Le graphe montre que la distribution normale ne peut jamais surpasser complètement la distribution de Cauchy, au voisinage de -10 .

Cependant, cette inexactitude n'affecte pas notre simulation, car la probabilité de cette réalisation est très faible ≈ 0003 .

- `cat()` : Cette fonction utilisée pour imprimer du texte ainsi que des variables ou des valeurs sur la console sans guillemets ou retours à la ligne supplémentaires.

Question 5

```
TH1<-function(U,V){
  NormaleSeq<-(sqrt(-2*log(U))*cos(2*pi*V))*sd
```

```

    return(NormaleSeq)
}
TH2<-function(U,V){
  NormaleSeq<-(sqrt(-2*log(U))*sin(2*pi*V))*sd
  return(NormaleSeq)
}

```

Nous avons développé 2 fonctions, *TH1* et *TH2*, basées sur la méthode de la transformation de Box-Muller.

Cette technique permet de générer des échantillons de variables aléatoires gaussiennes à partir de 2 échantillons uniformément distribués et indépendants dans l'intervalle $[0,1]$.

En multipliant le résultat par $sd = 3$, nous obtenons une réalisation de la loi normale $\mathcal{N}(0, 9)$.

La première fonction, *TH1*, utilise la formule $\sqrt{-2\log(U)} \times \cos(2\pi V)$, tandis que la seconde, *TH2*, utilise $\sqrt{-2\log(U)} \times \sin(2\pi V)$, où U et V sont des échantillons uniformément distribués et indépendants.

Ces fonctions produisent des échantillons indépendants suivant loi normale centrée réduite.

```

NormaleSeq<-TH1(UniSeq1[1:50],UniSeq1[51:100])
NormaleSeq<-c(NormaleSeq,TH2(UniSeq1[51:100],UniSeq1[1:50]))

```

Nous avons utilisé la séquence *UniSeq1*, que nous avons divisée en deux parties.

En appliquant ces deux parties aux formules de la transformation de Box-Muller, nous avons généré une séquence de variables aléatoires gaussiennes $\mathcal{N}(0, 9)$.

Il est important de noter que nous avons inversé l'ordre des parties de la séquence pour assurer une forte indépendance entre les réalisations.

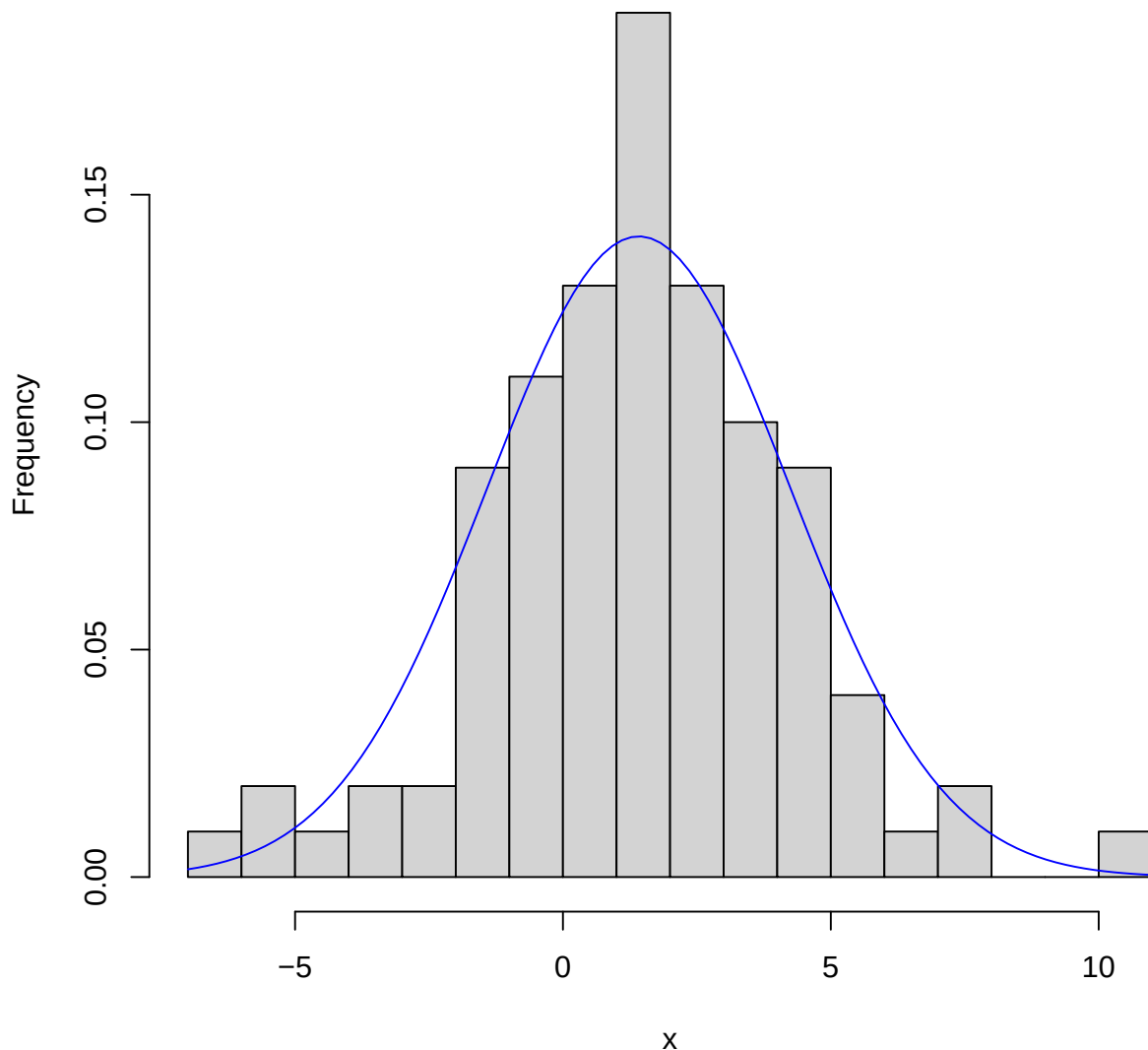
Cette inversion permet d'obtenir une indépendance maximale, car la même formule $\sqrt{-2\log(U)}$ est utilisée dans les deux étapes de la transformation de Box-Muller.

```

hist(NormaleSeq, breaks = 20, xlab = "x", ylab = "Frequency", freq = F)
curve(dnorm(x, mean = mean(NormaleSeq), sd = sd(NormaleSeq)), add = TRUE, col = "blue")

```

Histogram of NormaleSeq



```
(mean(NormaleSeq))  
## [1] 1.416118  
(sqrt(var(NormaleSeq)))  
## [1] 2.832497
```

on confirme de ce qu'on a générer de la loi normale $\mathcal{N}(0,9)$ est une vrai simulation car elle approche la distribution de la loi normale $\mathcal{N}(0,9)$ est on peut le remarquer de l'histogramme ici.

- `hist` : Trace un histogramme des valeurs de `NormaleSeq`.
L'argument `breaks` spécifie le nombre de classes dans l'histogramme.
Les étiquettes des axes x et y sont définies avec `xlab` et `ylab`, respectivement.
L'argument `freq = F` indique que l'histogramme doit être normalisé pour représenter une densité de probabilité plutôt que les fréquences brutes.
- `mean` : Calcule la moyenne des valeurs et l'imprime sur la console.
- `sqrt(var())` : Calcule l'écart-type des valeurs en prenant la racine carrée de la variance.
La variance est calculée avec `var()`.

Question 6

```
CauchySeqAccepte<-c()

for (i in 1:100) {
  if (M*dnorm(NormaleSeq[i],0,3)*UniSeq2[i]<=dcauchy(NormaleSeq[i],0,1)){
    CauchySeqAccepte<-c(CauchySeqAccepte,NormaleSeq[i])
  }
}
```

Si la condition est vraie (c'est-à-dire si l'échantillon 'NormaleSeq[i]' est accepté selon la méthode de rejet), alors 'NormaleSeq[i]' est ajouté à la séquence 'CauchySeqAccepte' à l'aide de la fonction 'c()'.

1. La boucle 'for' parcourt les entiers de 1 à 100 :
Cette boucle 'for' parcourt les entiers de 1 à 100. À chaque itération, elle examine un élément de la séquence 'NormaleSeq'.
2. La condition 'if' vérifie si la densité de la loi normale échantillonnée multipliée par une constante M et par une valeur uniformément distribuée $\text{UniSeq2}[i]$ est inférieure ou égale à la densité de la loi de Cauchy au même point.
3. Si la condition est vérifiée, la valeur courante de 'NormaleSeq' est ajoutée à la séquence 'CauchySeqAccepte'.

Question 7

```
X_n<-Seq1[length(Seq1)]
Y_n<-Seq2[length(Seq2)]
k<-0
while(length(CauchySeqAccepte)<100){
  x<-X_n
  X_n<-gcl(a,b,m)
  U_1<-X_n/(m-1)

  x<-X_n
  X_n<-gcl(a,b,m)
  U_2<-X_n/(m-1)

  x<-Y_n
  Y_n<-gcl(a,b,m)
  V_1<-Y_n/(m-1)

  x<-Y_n
  Y_n<-gcl(a,b,m)
  V_2<-Y_n/(m-1)

  C_X<-TH1(U_1,U_2)

  if (M*dnorm(C_X,0,3)*V_1<=dcauchy(C_X,0,1)){
    CauchySeqAccepte<-c(CauchySeqAccepte,C_X)
  }
}
```

```

C_X<-TH2(U_2,U_1)

if(M*dnorm(C_X,0,3)*V_2<=dcauchy(C_X,0,1)){
  CauchySeqAccepte<-c(CauchySeqAccepte,C_X)
}

k<-k+1 # c'est un compteur des itération du la boucle
}

```

Initialisation des variables:

$X_n \leftarrow \text{Seq1}[\text{length}(\text{Seq1})]$: X_n prend la valeur du dernier élément de la séquence Seq1.

$Y_n \leftarrow \text{Seq2}[\text{length}(\text{Seq2})]$: Y_n prend la valeur du dernier élément de la séquence Seq2.

$k \leftarrow 0$: k est initialisé à 0.

Dans la septième question, nous avons ajouté du code pour prolonger la génération de la même séquence. Tout d'abord, nous avons enregistré le dernier élément de la première séquence générée, X_n , en utilisant la syntaxe $X_n <- \text{Seq1}[\text{length}(\text{Seq1})]$.

Ensuite, nous avons fourni une explication du fonctionnement de la boucle avec un commentaire.

Boucle :

on génère 2 réalisation de loi uniforme pour chacune des séquence $UniSeq1$ et $UniSeq2$, on utilise c'est 2 réalisation $UniSeq1$ pour produire 2 autre de loi normale $\mathcal{N}(0,9)$ utilisant la transformation de Box Muller avec les deux fonction $TH1$ et $TH2$, après on utilise ces deux inégalité:

$$M \cdot dnorm(C_X, 0, 3) \cdot V_1 \leq f(dcauchy(C_X, 0, 1))$$

$$M \cdot dnorm(C_X, 0, 3) \cdot V_2 \leq f(dcauchy(C_X, 0, 1))$$

C_X est une variable utilisée pour stocker les deux réalisations de la loi normale $\mathcal{N}(0,9)$.

Cette boucle `while` continue tant que la longueur de la séquence `CauchySeqAccepte` est inférieure à 100.

À l'intérieur de cette boucle, vous devriez avoir du code qui utilise X_n , Y_n et k pour générer davantage d'échantillons de la loi de Cauchy par la méthode de rejet.

Les échantillons acceptés sont stockés dans la séquence `CauchySeqAccepte`.

La boucle `while` peut être utilisée pour continuer à générer des échantillons jusqu'à ce que la séquence atteigne la taille désirée (100 dans ce cas).

- `length()` : Cette fonction est utilisée pour obtenir la longueur d'un vecteur, d'une liste ou d'un facteur, Elle renvoie le nombre total d'éléments dans l'objet donné.

Question 8

```

K<-k*4+200 #200 c'est le nombre initial du realisation du gcl genere

```

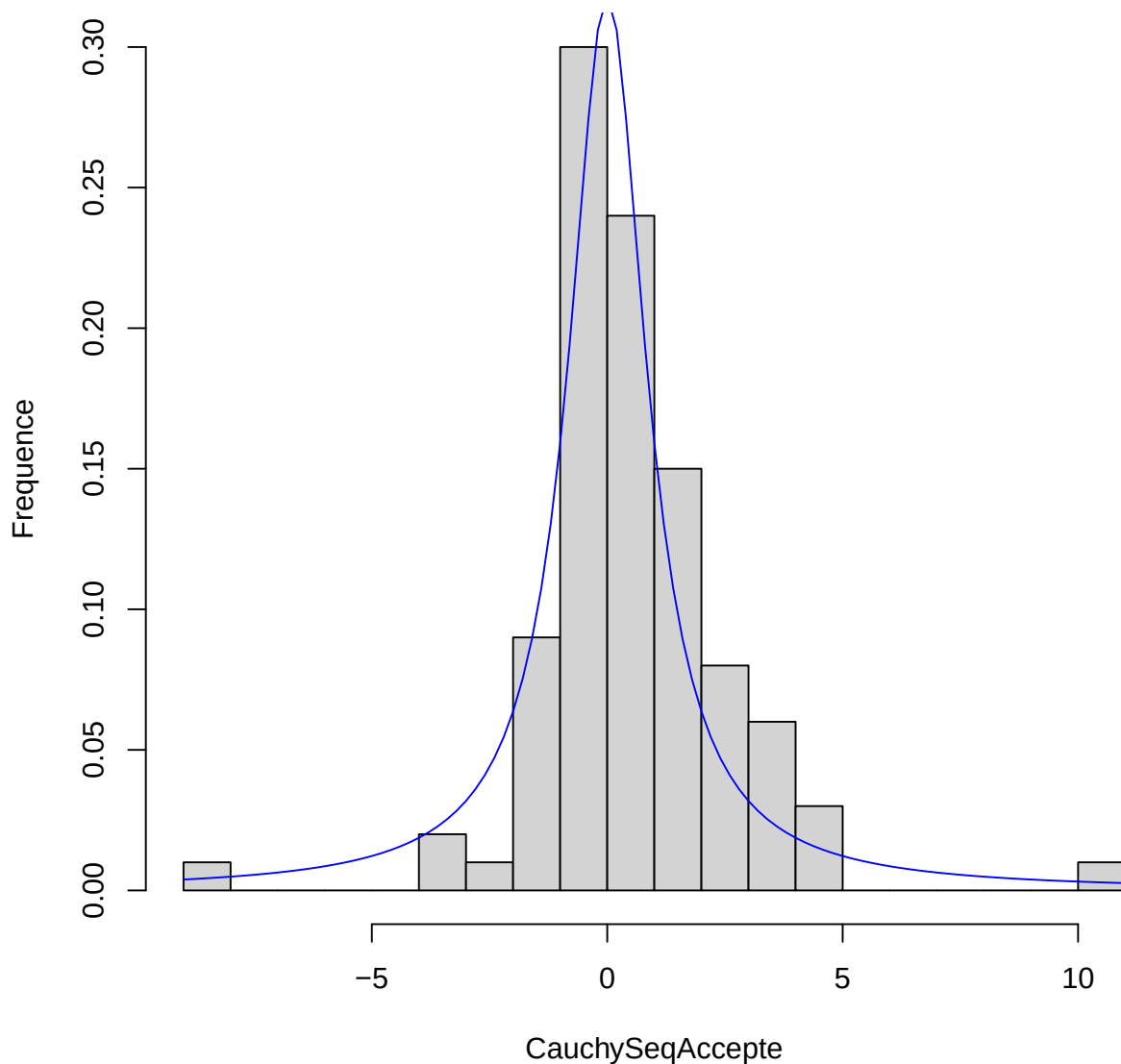
Nous avons multiplié cette variable par 4 car à chaque itération de la boucle, nous générons 4 réalisations du générateur de nombres pseudo-aléatoires.

```

hist(CauchySeqAccepte, breaks = 20, main = "Histogram du la loi du cauchy", ylab = "Frequence", freq =
curve(dcauchy(x,0,1), add = TRUE, col = "blue")

```

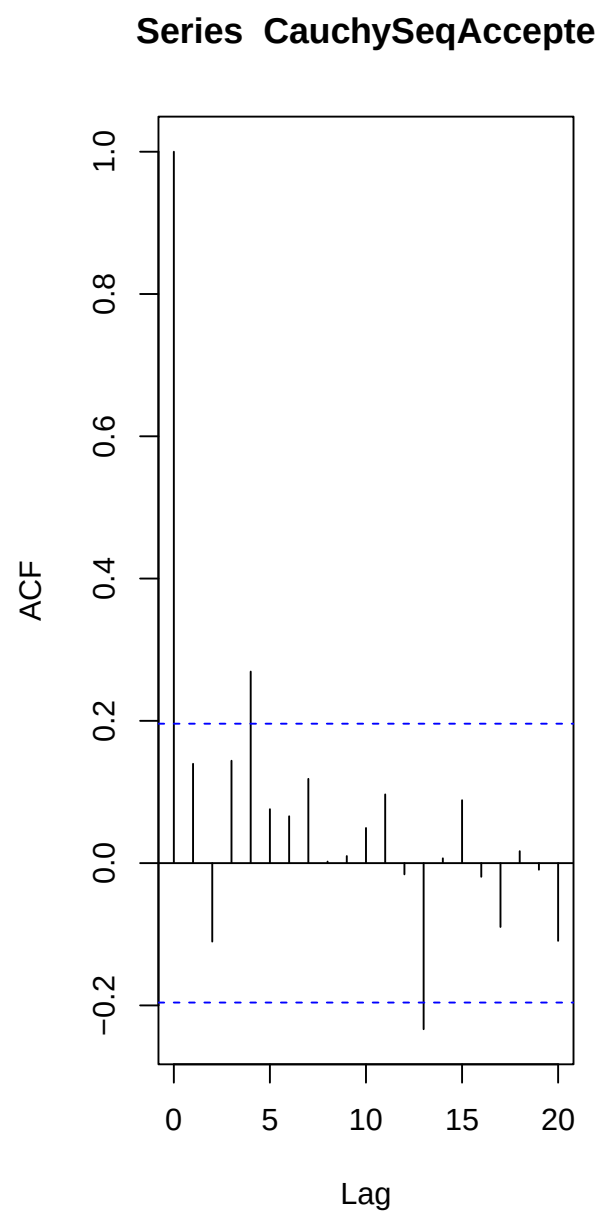
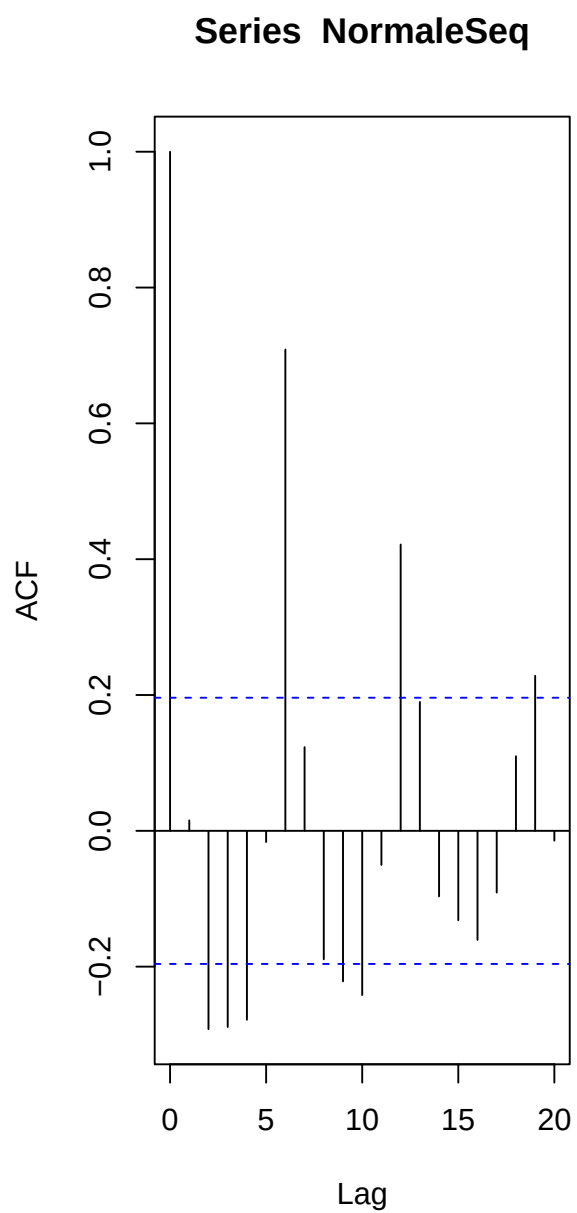
Histogramme de la loi de Cauchy



Nous observons que la séquence générée est proche de la loi de Cauchy, comme le montre le graphe, et si on augmente la taille de la séquence simulée elle se rapproche la distribution de Cauchy plus.

```
cat("le ratio du la taille du loi simule par rapport a le nombre du realisation du  
gcl est:",100/K)  
  
## le ratio du la taille du loi simule par rapport a le nombre du realisation du  
## gcl est: 0.2232143  
  
cat("le ratio du acceptance est",100/(k*2+100))  
  
## le ratio du acceptance est 0.4464286  
  
cat("Le nombre du realisation du gcl:",K)  
  
## Le nombre du realisation du gcl: 448
```

```
par(mfrow = c(1, 2))
acf(NormaleSeq)
acf(CauchySeqAccepte)
```



- **acf** : Cette fonction calcule et trace la fonction d'autocorrélation d'une série de données.

Conclusion:

Ce TP a permis de mettre en pratique la simulation de distributions de probabilité en utilisant la méthode de rejet.

Nous avons montré qu'il est possible de générer des échantillons de la loi de *Cauchy*(0, 1) à partir d'échantillons de la loi normale $\mathcal{N}(0, 9)$.

Mais malgré certaines limitations, nous constatons à partir du ratio que nous avons obtenu à partir du code R que nous avons accepté environ 47% des réalisations de la *normale* comme réalisations de *Cauchy*, ce qui dans ce cas est un bon pourcentage, mais il existe certaines limitations comme la distribution *normale* ne peut jamais être complètement plus grande que la distribution de *Cauchy*, ce qui peut entraîner une certaine inexactitude.

Un autre inconvénient de la méthode de rejet est que nous avons besoin de 448 réalisations du GCL pour obtenir 100 réalisations de *Cauchy*(0, 1), ce qui est environ 4 fois la taille de la séquence voulue à l'échelle plus grande, cela peut poser des problèmes computationnels et comme nous pouvons le voir à partir du graphique d'autocorrélation, les réalisations simulées sont un peu corrélées, ce qui est un peu mauvais mais ce problème est résolu si nous augmentons la taille simulée.

En dehors de tout cela, la méthode de rejet est efficace et vraiment bonne pour l'échantillonnage et très utilisable, simple, facile et pratique pour simuler des distributions de probabilité et nous pouvons l'utiliser pour résoudre de nombreux problèmes tels que les distributions de probabilité complexes et le calcul d'intégrales, ce dont vous aurez besoin un jour en tant que statisticien ou mathématicien.

Code R

```
1 gcl<-function(a,b,m){
2   x<-(a*x+b)%m
3
4   return(x)
5 }
6
7
8 m=1171*8*7*9
9 a=1171*4*7*3+1
10 b=11*13*17*19
11
12 x<-789547
13 Seq1<-replicate(100,gcl(a,b,m))
14 UniSeq1<-Seq1/(m-1)
15 x<-25794
16 Seq2<-replicate(100,gcl(a,b,m))
17 UniSeq2<-Seq2/(m-1)
18
19 x<-25794
20 Seq2<-replicate(100,gcl(a,b,m))
21
22 curve(dnorm(x,mean = 0,sd=1), from = -20, to = 20, col = "purple", lwd = 1)
23
24 curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
25 legend("topright", legend = c("Densite du la loi Normale", "Densite du la loi Cauchy"),
26       col = c("purple", "blue"), lty = 1, lwd = 1)
27
28 (MaxNormal<-dnorm(0, mean = 0,sd=1))
29 (MaxCauchy<-dcauchy(0,0,1))
30
31 sd=3
32 curve(dnorm(x,mean = 0,sd=1), from = -20, to = 20, col = "purple", lwd = 1)
33
34 curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
35 curve(dnorm(x,mean = 0,sd=sd), from = -20,add = TRUE, to = 20, col = "red", lwd = 1)
36 legend("topright", legend = c("Densite du la loi Normale", "Densite du la loi Cauchy",
37   "Densite du la loi Normale sigma=3"),
38       col = c("purple", "blue","red"), lty = 1, lwd = 1)
39 (MaxNormal3<-dnorm(0,0,3))
40
41 (MaxNormal3>=MaxCauchy)
42
43
44 M=1
45 while (M*MaxNormal3<=MaxCauchy) {
46
47   M<-M+0.1
48
49 }
50 cat("la constante est egal:",M)
51
52 (M*MaxNormal3>=MaxCauchy)
53
54
55 curve(M*dnorm(x,mean = 0,sd=3), from = -20, to = 20, col = "purple", lwd = 1)
56
57 curve(dcauchy(x,0,1), from = -20, to = 20,add = TRUE, col = "blue", lwd = 1)
58 legend("topright", legend = c("Densite du la loi Normale sigma=3 multiply par 2.4",
59   "Densite du la loi Cauchy"),
60       col = c("purple", "blue"), lty = 1, lwd = 1)
61
62 TH1<-function(U,V){
```

```

63  NormaleSeq<-(sqrt(-2*log(U))*cos(2*pi*V))*sd
64  return(NormaleSeq)
65  }
66  TH2<-function(U,V){
67    NormaleSeq<-(sqrt(-2*log(U))*sin(2*pi*V))*sd
68    return(NormaleSeq)
69  }
70
71  NormaleSeq<-TH1(UniSeq1[1:50],UniSeq1[51:100])
72  NormaleSeq<-c(NormaleSeq,TH2(UniSeq1[51:100],UniSeq1[1:50]))
73
74
75  hist(NormaleSeq, breaks = 20, xlab = "x", ylab = "Frequency", freq = F)
76  curve(dnorm(x, mean = mean(NormaleSeq), sd = sd(NormaleSeq)), add = TRUE, col = "blue")
77  (mean(NormaleSeq))
78  (sqrt(var(NormaleSeq)))
79
80  CauchySeqAccepte<-c()
81
82
83  for (i in 1:100) {
84    if(M*dnorm(NormaleSeq[i],0,3)*UniSeq2[i]<=dcauchy(NormaleSeq[i],0,1)){
85      CauchySeqAccepte<-c(CauchySeqAccepte,NormaleSeq[i])
86    }
87  }
88
89  X_n<-Seq1[length(Seq1)]
90  Y_n<-Seq2[length(Seq2)]
91  k<-0
92  while(length(CauchySeqAccepte)<100){
93    x<-X_n
94    X_n<-gcl(a,b,m)
95    U_1<-X_n/(m-1)
96
97    x<-X_n
98    X_n<-gcl(a,b,m)
99    U_2<-X_n/(m-1)
100
101    x<-Y_n
102
103    Y_n<-gcl(a,b,m)
104
105    V_1<-Y_n/(m-1)
106
107    x<-Y_n
108
109    Y_n<-gcl(a,b,m)
110
111    V_2<-Y_n/(m-1)
112
113    C_X<-TH1(U_1,U_2)
114
115    if(M*dnorm(C_X,0,3)*V_1<=dcauchy(C_X,0,1)){
116      CauchySeqAccepte<-c(CauchySeqAccepte,C_X)
117    }
118
119    C_X<-TH2(U_2,U_1)
120
121    if(M*dnorm(C_X,0,3)*V_2<=dcauchy(C_X,0,1)){
122      CauchySeqAccepte<-c(CauchySeqAccepte,C_X)
123    }
124
125    k<-k+1
126
127
128  }

```

```
129
130
131 K<-k*4+200 #200 c'est le nombre initial du realisation du gcl genere
132 hist(CauchySeqAccepte, breaks = 20, main = "Histogram du la loi du cauchy", ylab = "
    Frequence", freq = F)
133 curve(dcauchy(x,0,1), add = TRUE, col = "blue")
134 acf(CauchySeqAccepte)
135 acf(NormleSeq)
136
137 cat("le ratio d'acceptation est:",100/K)
138
139
140 cat("Le nombre du realisation du gcl:",K)
```